

NORA — Network Oracle for Reliable Answers

Dokumen Teknis Lengkap

Atribut	Detail
Dokumen	NORA Technical Overview
Versi	1.0
Tanggal	Juni 2026
Disiapkan oleh	Tim ArahKarya
Kolaborasi	PT Arah Karya Sinergi x NOZ (Tri Sumarno)
Status	Final — siap render PDF

Ringkasan Eksekutif

NORA (Network Oracle for Reliable Answers) adalah platform SaaS *AI Research Engine* multi-topik yang dirancang khusus untuk domain standar telekomunikasi. NORA menjawab pertanyaan teknis engineer telco dengan jawaban yang **tergrounded pada spec resmi** — bukan dari ingatan model — sehingga risiko halusinasi ditekan hingga nol.

Setiap jawaban NORA dilengkapi:

- **Confidence score** (0–1) sebagai ukuran keyakinan sistem.
- **Referensi sumber** per nomor section spec (mis. §4.7.3.1).
- **Verdict verifikasi** (VALID / PARTIAL / INVALID) dari model kedua yang independen.

NORA dibangun sebagai **kolaborasi teknis NOZ (Tri Sumarno) x PT Arah Karya Sinergi**, dengan arsitektur multi-tenant yang siap scale ke banyak pengguna paralel. Platform saat ini berjalan secara **live** di nora.arahkarya.com.

1. Apa Itu NORA & Masalah yang Dipecahkan

1.1 Masalah

Engineer telekomunikasi menghadapi tantangan nyata saat harus merujuk standar teknis:

Masalah	Dampak
Spec 3GPP ratusan halaman (TS 24.008 ~600+ hal., 257 versi rilis)	Waktu riset lama, melelahkan
Versi spec berubah tiap rilis (R98 s.d. R18)	Rawan salah interpretasi antar-versi

Masalah	Dampak
LLM umum menjawab dari <i>parametric memory</i>	Halusinasi pada detail teknis (nomor section, IE, prosedur) — tidak dapat diterima untuk domain regulasi
Tidak ada jejak audit	Jawaban hilang, tidak bisa ditelusuri

1.2 Solusi NORA

NORA mengingesti **spec resmi** ke vector database, lalu menjawab melalui pipeline **Retrieval-Augmented Generation (RAG) + dual-model + validation layer**:

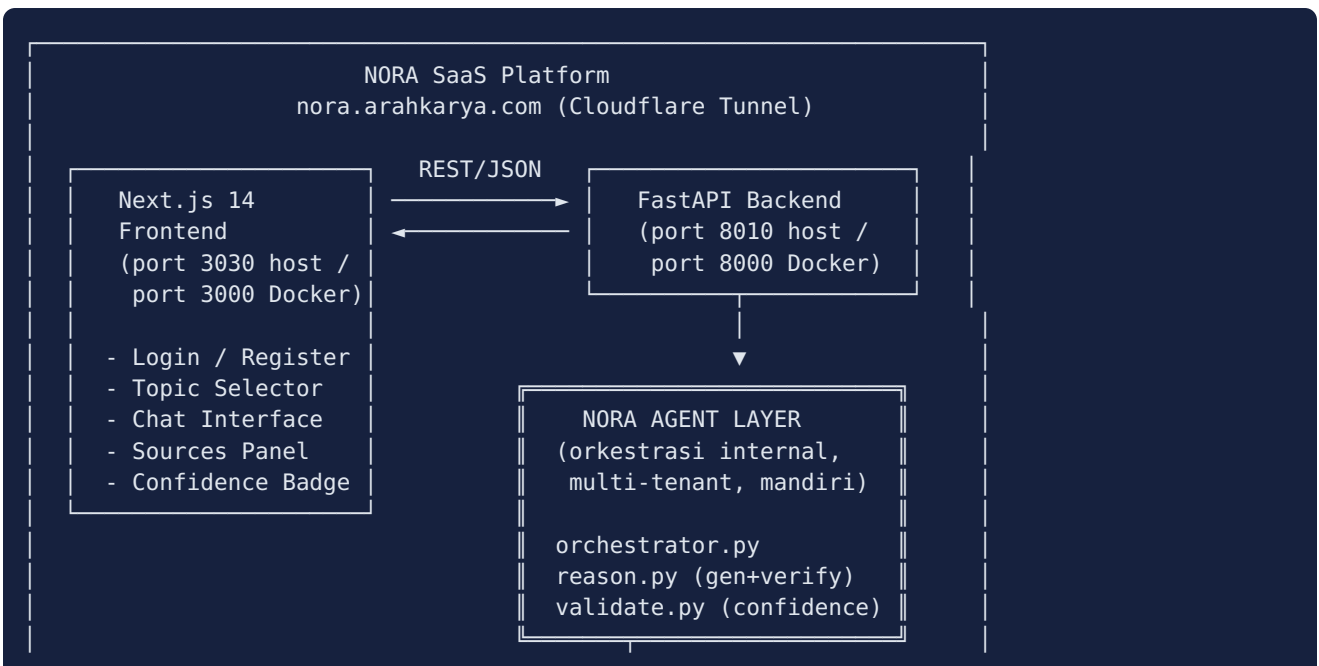
1. Sumber jawaban selalu spec asli yang diindeks — bukan ingatan model.
2. Model **Generator** menyusun jawaban dari konteks yang di-retrieve.
3. Model **Verifier** independen memverifikasi setiap klaim sebelum jawaban dikirim ke user.
4. Jika tidak ada konteks relevan → sistem menjawab "*tidak ditemukan dalam spec*" dan menolak mengarang.

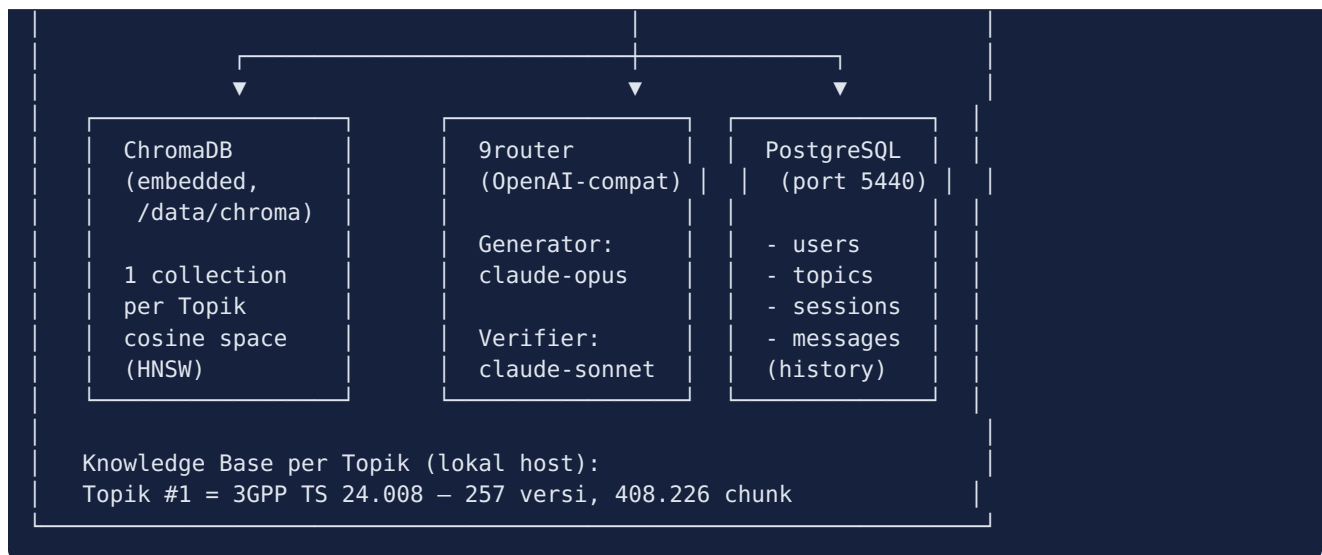
1.3 Proposisi Nilai Inti

#	Nilai	Penjelasan
1	Anti-halusinasi	Jawaban wajib bersumber dari spec resmi yang terindeks
2	Self-validating	Model kedua (Verifier) mengecek jawaban model pertama (Generator)
3	Auditable	Setiap jawaban punya confidence score + sumber yang dapat ditelusuri
4	Multi-topik & extensible	Satu engine RAG, banyak knowledge base; topik baru tidak perlu ubah kode engine
5	Engine-agnostic	Cloud (9router) atau lokal (Ollama), satu arsitektur

2. Arsitektur Sistem

2.1 Diagram Layer





2.2 Penjelasan Tiap Komponen

Frontend — Next.js 14

- Framework React dengan Server-Side Rendering (SSR).
- Berjalan di port `3030` (host) / `3000` (Docker container).
- Fitur utama: halaman login/register, topic selector, chat interface, sources panel expandable, confidence badge.
- Semua request `/api/*` di-proxy ke backend internal melalui Next.js rewrites — satu origin, cookie aman, tanpa CORS issue.
- File: `frontend/`

Backend — FastAPI

- API layer Python async, berjalan di port `8010` (host) / `8000` (Docker container).
- Bertanggung jawab atas: autentikasi, routing request, persistensi ke PostgreSQL, dan pemanggilan NORA Agent Layer sebagai service internal.
- Routes:
 - `POST /api/auth/register` — registrasi user baru
 - `POST /api/auth/login` — login, set JWT cookie
 - `GET /api/auth/me` — info user aktif
 - `GET /api/topics` — daftar topik tersedia
 - `POST /api/query` — kirim pertanyaan, dapatkan jawaban RAG
 - `GET /api/history` — riwayat chat per-user
- Swagger/ReDoc dimatikan di produksi (`NORA_ENABLE_DOCS=false` default) untuk mengurangi attack surface.
- File: `backend/nora/main.py`, `backend/nora/api/routes_*.py`

NORA Agent Layer — Orkestrasi Internal

- Engine orkestrasi mandiri milik NORA, ditulis sebagai kode Python murni (bukan runtime generik eksternal).
- Dipanggil backend sebagai library/service internal — stateless per-request, state durable di PostgreSQL.

- Menjalankan loop: **retrieve** → **generate** → **verify** → **validate** → **loop jika INVALID**.
- Multi-tenant: setiap request terisolasi per-user; memory dan history tersimpan di Postgres, bukan memori global.
- File: `backend/nora/pipeline/orchestrator.py` , `backend/nora/pipeline/reason.py` , `backend/nora/pipeline/validate.py`

ChromaDB — Vector Store

- Embedded (tidak berjalan sebagai service terpisah), disimpan di `backend/data/chroma` (di-mount ke `/app/data/chroma` di container).
- Satu collection per Topik (isolasi data penuh antar-Topik).
- Menggunakan **cosine similarity** dengan indeks HNSW.
- Embedding disuplai oleh sistem (bukan ChromaDB yang generate) — konsisten antara ingest dan query.
- File: `backend/nora/rag/store.py`

PostgreSQL — State & Auth

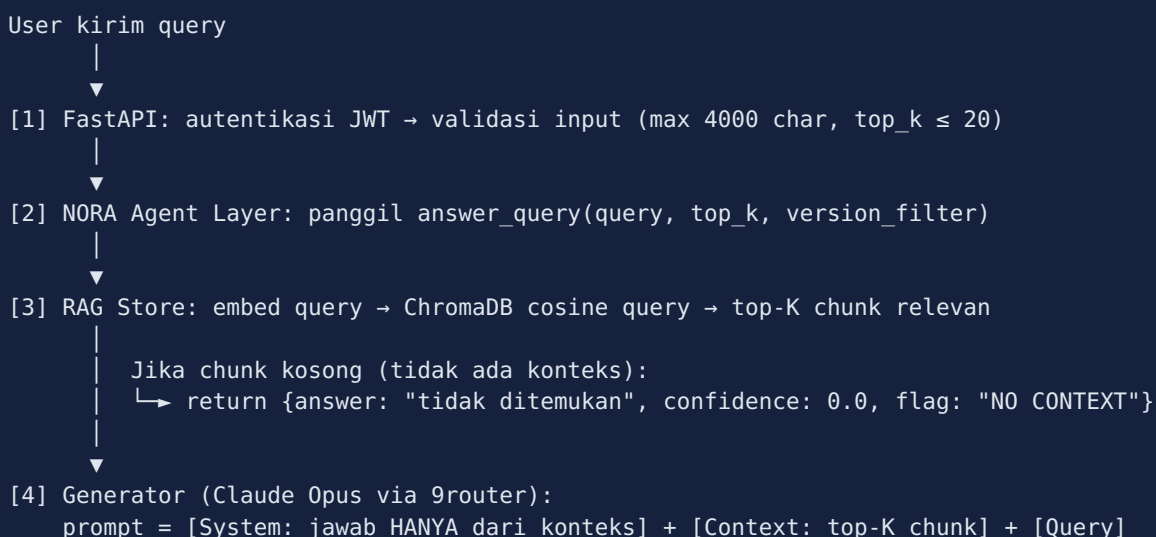
- Berjalan di port `5440` (host) / `5432` (internal Docker network).
- Menyimpan: tabel `users` , `topics` , `chat_sessions` , `messages` (dengan `sources_json` dan `confidence`).
- Dijaga dengan healthcheck sebelum backend start.
- File: `backend/nora/db/models.py`

9router — LLM & Embedding Gateway

- OpenAI-compatible API gateway yang berjalan di `localhost:20128` (host RPi5, diakses backend via `host.docker.internal`).
- Melayani dua model sekaligus: Generator (Opus) dan Verifier (Sonnet).
- Juga melayani endpoint embedding Gemini (`gemini-embedding-001`).

3. Alur Kerja RAG End-to-End

3.1 Diagram Alur



```

→ jawaban awal
|
▼
[5] Verifier (Claude Sonnet via 9router):
prompt = [Context] + [Jawaban awal]
→ output JSON: {verdict: VALID|PARTIAL|INVALID, reason, unsupported_claims}
|
|   Jika INVALID (maks 2x loop):
|   └─ [4'] Generator re-generate dengan hint dari Verifier
|       └─ [5'] Verifier re-check → lanjut ke [6]
|
▼
[6] Validate: hitung confidence score
score = (0.6 × verdict_weight) + (0.4 × avg_similarity) - penalty_unsupported
flag = "LOW CONFIDENCE" jika score < 0.7
|
▼
[7] Susun output:
{answer, confidence, flag, verifier_verdict, verifier_reason,
 regeneration_count, sources[], query_id}
|
▼
[8] FastAPI: persist ke PostgreSQL (ChatSession + Message user + Message assistant)
|
▼
[9] Response JSON → Next.js → tampil di UI (jawaban + badge confidence + sumber)

```

3.2 Langkah Bernomor (Deskriptif)

Langkah	Proses	Komponen
1	User mengirim pertanyaan melalui chat UI	Frontend → FastAPI
2	Backend memvalidasi JWT dan input, lalu memanggil Agent Layer	<code>routes_query.py</code>
3	Agent Layer meng-embed query → mencari top-K chunk paling relevan dari ChromaDB (cosine similarity)	<code>orchestrator.py</code> , <code>store.py</code>
4	Jika tidak ada chunk relevan, sistem langsung mengembalikan "tidak ditemukan" tanpa memanggil LLM	<code>orchestrator.py</code>
5	Generator (Claude Opus) menyusun jawaban berdasarkan konteks chunk yang di-retrieve	<code>reason.py</code> — <code>generate()</code>
6	Verifier (Claude Sonnet) menilai apakah semua klaim jawaban didukung konteks → VALID / PARTIAL / INVALID	<code>reason.py</code> — <code>verify()</code>
7	Jika INVALID, Generator melakukan regenerasi dengan hint dari Verifier (maks. 2 kali)	<code>reason.py</code> — <code>generate_verified()</code>
8	Confidence score dihitung dari: bobot verdict + rata-rata similarity + penalti klaim tidak bersumber	<code>validate.py</code> — <code>confidence()</code>
9	Output lengkap dikembalikan ke user; percakapan disimpan ke PostgreSQL	<code>orchestrator.py</code> , <code>routes_query.py</code>

3.3 Fallback Chain LLM

NORA mengimplementasi fallback otomatis jika model utama overload atau timeout:

Peran	Model Utama	Fallback 1	Fallback 2
Generator	cc/claude-opus-4-8	cc/claude-sonnet-4-6	ag/gemini-3-flash
Verifier	cc/claude-sonnet-4-6	ag/gemini-3-flash	—

Error transient yang memicu fallback: HTTP 429, 500, 502, 503, 529, timeout, overload.

4. Embedding & Vector Store

4.1 Multi-Backend Embedding

NORA mendukung tiga backend embedding yang dapat dipilih via environment variable `NORA_EMBED_BACKEND`. Backend yang dipakai saat ingest **harus sama** dengan saat query — dimensi vektor harus konsisten.

```
NORA_EMBED_BACKEND=9router # default – Gemini cloud via 9router
NORA_EMBED_BACKEND=ollama # Ollama lokal (offline)
NORA_EMBED_BACKEND=local # fastembed ONNX CPU (paling ringan)
```

4.2 Tabel Perbandingan Backend Embedding

Backend	Provider	Model Default	Dimensi Vektor	Mode	Keterangan
9router (default)	Google / 9router	gemini/gemini-embedding-001	3072	Cloud	Kualitas terbaik; RAM RPi5 aman (komputasi di cloud)
ollama	Ollama (lokal)	nomic-embed-text	768	Lokal offline	Privasi penuh; mxbai-embed-large = 1024 dim
local	fastembed ONNX	BAAI/bge-small-en-v1.5	384	Lokal offline CPU	Paling ringan; cocok resource terbatas

Peringatan: Ganti backend embedding = wajib re-ingest seluruh knowledge base. Index dan query harus memakai backend + model yang sama.

4.3 ChromaDB Vector Store

- **Tipe:** Embedded (tidak ada service terpisah), persisten di disk.
- **Path:** backend/data/chroma (host) → /app/data/chroma (container).
- **Konfigurasi env:** NORA_CHROMA_PATH , NORA_COLLECTION .
- **Similarity metric:** Cosine (HNSW space).
- **Struktur collection:** 1 collection per Topik, nama diambil dari field collection_name di tabel topics .
- **Topik #1:** collection ts24008 , berisi seluruh chunk 3GPP TS 24.008.

4.4 Proses Ingest ke ChromaDB

```
# backend/nora/rag/store.py — add_chunks()
col.upsert(ids=..., embeddings=embed_texts(texts), documents=texts, metadatas=metas)
```

- Batch size 32 teks per call ke embedding API (aman di bawah limit Gemini).
- Deduplication: jika `chroma_id` sama (section muncul dua kali), suffix counter ditambahkan.
- `upsert` — aman untuk re-ingest (tidak duplikat).

5. Knowledge Base & Chunking

5.1 Knowledge Base Topik #1: 3GPP TS 24.008

3GPP TS 24.008 adalah spesifikasi *Mobile Radio Interface Layer 3* yang mendefinisikan protokol inti jaringan 2G/3G/4G: **Mobility Management (MM)**, **GPRS Mobility Management (GMM)**, **Call Control (CC)**, dan **Session Management (SM)**.

Item	Detail
Spec	TS 24.008 — Mobile radio interface Layer 3; Core network protocols; Stage 3
Cakupan	MM / GMM / CC / SM
Jumlah versi	257 rilis (R98 v3.0.0 → R18 j-series, tahun 1999–2026)
Format mentah	<code>.doc</code> / <code>.docx</code> (per zip dari 3GPP FTP) — total ~778 MB
Format diolah	<code>.txt</code> via LibreOffice headless — ~229 MB
Lokasi host	<code>~/data/3gpp/24008/txt</code>
Total chunk terindeks	~ 408.226 chunk (rata-rata ~1.051 chunk/versi)
Collection ChromaDB	<code>ts24008</code>

5.2 Section-Aware Chunking

File `.txt` hasil konversi LibreOffice dipecah berdasarkan **nomor section 3GPP** (bukan ukuran fixed), sesuai PRD FR-ING-002.

Regex pendeteksi heading section:

```
SECTION_RE = re.compile(
    r'^\s*(?P<num>\d{1,2}(?:\.\d{1,3}){0,5})\s*(\t| )+(?P<title>[A-Z][^\n]{2,120})\s*$'
)
```

Contoh heading yang terdeteksi: `4.7.3 GPRS attach procedure`

Parameter chunking:

Parameter	Nilai	Keterangan
<code>MAX_CHARS</code>	3200 char (~800 token)	Batas maksimum satu chunk
<code>OVERLAP_CHARS</code>	200 char	Overlap antar sub-chunk jika section terlalu panjang

Alur chunking per file:

1. Baca file `.txt`, skip blok TOC (deteksi dengan pola `\t<angka>` di akhir baris).
2. Parse heading section dengan `SECTION_RE` → dapatkan `(section_num, section_title, body_lines)`.
3. Jika body section `> MAX_CHARS`, sub-split dengan overlap 200 char.
4. Tiap chunk diberi metadata wajib.

Metadata tiap chunk:

```
{
  "spec_id":      "TS 24.008",
  "version":      "g50",          # kode rilis 3GPP dari nama file
  "section":      "4.7.3",
  "section_title": "GPRS attach procedure",
  "source_file":  "24008-g50.txt",
  "chunk_index":  0              # indeks sub-chunk jika section di-split
}
```

Chroma ID format:

```
TS_24.008|g50|4.7.3|0
```

6. Multi-Topik & Skalabilitas

6.1 Konsep Topik

NORA adalah **engine RAG generik** yang melayani banyak knowledge base. Tiap knowledge base disebut **Topik**. Tiap Topik memiliki collection ChromaDB sendiri namun berbagi seluruh pipeline RAG yang sama.

Model data Topik (PostgreSQL, tabel `topics`):

```
class Topic(Base):
    id          # UUID hex
    slug        # mis. "3gpp-ts24008" — unik, dipakai sebagai key
    name        # mis. "3GPP TS 24.008"
    description  # ringkasan domain
    collection_name # nama collection di ChromaDB: "ts24008"
    created_at
```

6.2 Roadmap Topik

Topik	Status	Domain
3GPP TS 24.008	● Live (terbukti E2E)	NAS Layer 3 — MM / GMM / CC / SM
3GPP TS 23.501	◦ Planned	5G System Architecture
3GPP TS 33.501	◦ Planned	5G Security
ITU-T (seri Q/X)	◦ Planned	Signalling & data networks
IEEE 802.x	◦ Planned	LAN / Wireless

Topik	Status	Domain
GSMA / O-RAN	◦ Planned	Operator & Open RAN

6.3 Alur Penambahan Topik Baru

```

[Admin] registrasi metadata Topik (slug, name, description, collection_name)
↓
Sistem seed ke tabel topics (PostgreSQL)
↓
[Admin] jalankan ingest: point ke folder .txt knowledge base Topik
↓
Chunker section-aware memproses file → embed → upsert ke ChromaDB collection baru
↓
Topik status → "live" — siap dipakai user, tanpa ubah kode engine

```

6.4 Skalabilitas Arsitektur

Aspek	Pendekatan
Multi-tenant	Stateless per-request; state (memory, history) di Postgres — aman di-load-balance
Isolasi data	Tiap Topik = collection ChromaDB sendiri; retrieval tidak pernah lintas-collection
Scale horizontal	Backend FastAPI dapat di-replikasi; state tidak di-memory proses
Upgrade vector store	Demo: ChromaDB embedded → Produksi: swap ke Qdrant tanpa ubah arsitektur
RAM guard	Container backend dibatasi <code>mem_limit: 3g</code> (Docker Compose) — cegah OOM RPI5

7. Keamanan & Auth

7.1 Autentikasi

Mekanisme	Detail
Password hashing	<code>bcrypt</code> via <code>passlib</code> — hash one-way, tidak pernah simpan plaintext
Token	JWT (JSON Web Token), algoritma HS256, masa berlaku 7 hari (<code>COOKIE_MAX_AGE = 7 * 24 * 3600</code>)
Cookie	<code>httpOnly=True</code> (tidak bisa diakses JavaScript), <code>SameSite=lax</code> , <code>Secure=True</code> di produksi
Secret key	<code>SECRET_KEY</code> via environment variable — aplikasi fail-fast jika tidak di-set di produksi

Flow autentikasi:

```

POST /api/auth/register → bcrypt hash password → simpan ke users (Postgres)
POST /api/auth/login    → verifikasi bcrypt → buat JWT → set httpOnly cookie
GET  /api/*             → middleware baca cookie → decode JWT → inject user ke route
POST /api/auth/logout   → delete cookie

```

7.2 Input Validation

Parameter	Batasan
<code>message</code> (query)	Minimal 1 karakter, maksimal 4000 karakter
<code>top_k</code>	Minimal 1, maksimal 20
<code>topic_id</code>	Wajib ada, divalidasi ke tabel <code>topics</code>
<code>session_id</code>	Jika disuplai, harus milik user yang sama (cek <code>user_id</code>)

7.3 Security Headers

Konfigurasi produksi mengaktifkan security headers standar:

Header	Nilai	Tujuan
<code>Strict-Transport-Security</code>	<code>max-age=31536000; includeSubDomains</code>	HSTS — paksa HTTPS
<code>Content-Security-Policy</code>	Dibatasi ke origin yang diizinkan	Cegah XSS
<code>X-Frame-Options</code>	<code>DENY</code>	Cegah clickjacking
<code>X-Content-Type-Options</code>	<code>nosniff</code>	Cegah MIME sniffing

7.4 Akses Remote

Akses publik ke `nora.arahkarya.com` melalui **Cloudflare Tunnel** — tidak ada port yang terbuka langsung ke internet di host RPi5. Semua traffic dienkripsi dan difilter oleh Cloudflare sebelum sampai ke server.

7.5 CORS

Origins yang diizinkan dikonfigurasi via `NORA_CORS_ORIGINS` :

```
NORA_CORS_ORIGINS=http://localhost:3030,https://nora.arahkarya.com
```

8. Deployment

8.1 Stack Docker Compose

NORA dijalankan dengan tiga service Docker Compose:

```
# docker-compose.yml
services:
  postgres: # state & auth
  backend: # FastAPI + ChromaDB embedded
  frontend: # Next.js 14
```

Port mapping:

Service	Port Host	Port Container	Keterangan
<code>frontend</code>	<code>3030</code>	<code>3000</code>	Next.js dashboard

Service	Port Host	Port Container	Keterangan
backend	8010	8000	FastAPI API
postgres	5440	5432	PostgreSQL 16
ChromaDB	—	—	Embedded di backend, tidak expose port

Jaringan internal:

nora-net (bridge) – subnet 10.0.14.0/24

Seluruh service berkomunikasi lewat network `nora-net`. Backend mengakses 9router dan Ollama di host melalui `host.docker.internal`.

8.2 Quickstart

```
# Clone & masuk ke direktori
cd /home/yay/apps/nora

# Salin dan edit konfigurasi
cp .env.example .env
# Edit: SECRET_KEY, POSTGRES_PASSWORD, NORA_ROUTER_URL, dll.

# Build dan jalankan semua service
docker compose up -d --build

# Cek status
docker compose ps
docker compose logs -f backend
```

8.3 Environment Variables Utama

Variabel	Default	Keterangan
<code>SECRET_KEY</code>	<i>(wajib di-set)</i>	Secret JWT — fail-fast jika dev default di produksi
<code>DATABASE_URL</code>	<code>postgresql+psycopg://nora:...@postgres:5432/nora</code>	Koneksi PostgreSQL
<code>NORA_ENGINE</code>	<code>9router</code>	Engine LLM: <code>9router</code> <code>ollama</code>
<code>NORA_GEN_MODEL</code>	<code>cc/claude-opus-4-8</code>	Model generator
<code>NORA_VERIFY_MODEL</code>	<code>cc/claude-sonnet-4-6</code>	Model verifier
<code>NORA_EMBED_BACKEND</code>	<code>9router</code>	Backend embedding: <code>9router</code> <code>ollama</code> <code>local</code>
<code>NORA_EMBED_MODEL</code>	<code>gemini/gemini-embedding-001</code>	Model embedding (dim 3072 untuk 9router)
<code>NORA_ROUTER_URL</code>	<code>http://host.docker.internal:20128/v1</code>	URL 9router (OpenAI-compatible)
<code>NORA_CHROMA_PATH</code>	<code>/app/data/chroma</code>	Path ChromaDB di dalam container
<code>NORA_COLLECTION</code>	<code>ts24008</code>	Collection ChromaDB default
<code>NORA_CORS_ORIGINS</code>	<code>http://localhost:3030,https://nora.arahkarya.com</code>	Origins CORS yang diizinkan

Variabel	Default	Keterangan
<code>NORA_COOKIE_SECURE</code>	<code>true</code>	Set <code>false</code> untuk development HTTP
<code>NORA_ENABLE_DOCS</code>	<code>false</code>	Set <code>true</code> untuk aktifkan Swagger/ ReDoc

8.4 Ingest Knowledge Base

```
# Masuk ke environment backend
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Test ingest 1 versi (verifikasi pipeline)
python -m nora.ingest.run --limit 1

# Ingest semua 257 versi TS 24.008
python -m nora.ingest.run

# Cek statistik indeks
python -c "from nora.rag.store import stats; print(stats())"
```

8.5 Cloudflare Tunnel

Akses publik NORA menggunakan Cloudflare Tunnel — tidak ada port yang dibuka ke internet:

```
Internet → Cloudflare Edge → Tunnel → host RPi5 → nora-frontend:3030
                                     ↗ /api/* → nora-backend:8010
```

- Domain: `nora.arahkarya.com`
- Frontend (port `3030`) melayani semua request; path `/api/*` di-proxy ke backend internal via Next.js rewrites.
- Cookie `Secure=true` karena semua traffic masuk via HTTPS Cloudflare.

8.6 Topologi Produksi vs Demo

Aspek	Demo (RPi5 saat ini)	Produksi (target)
Vector store	ChromaDB embedded	Qdrant (swap via config, tanpa rewrite)
LLM	9router cloud (Opus/Sonnet)	9router cloud atau vLLM self-hosted
Backend replika	1 instance	N replika + load balancer
Database	PostgreSQL container	PostgreSQL managed / dedicated
Hardware	Raspberry Pi 5 (8GB)	VPS / cloud server

Lampiran A: Struktur Repositori

```
nora/
├── docs/
│   └── BRD-NORA.md          # Business Requirements Document
```

```

├── PRD-NORA.md          # Product Requirements Document
├── NORA-Overview.md     # Dokumen ini
├── screenshots/         # Tampilan UI (login, workspace, jawaban)
├── backend/
│   ├── nora/
│   │   ├── main.py      # FastAPI app entrypoint, startup seed
│   │   ├── api/
│   │   │   ├── routes_auth.py    # Register, login, logout, me
│   │   │   ├── routes_query.py   # POST /api/query – main RAG endpoint
│   │   │   ├── routes_topics.py  # GET /api/topics
│   │   │   └── routes_history.py # GET /api/history
│   │   ├── pipeline/
│   │   │   ├── orchestrator.py   # NORA Agent Layer: alur utama RAG
│   │   │   ├── reason.py         # Generator + Verifier + fallback chain
│   │   │   └── validate.py       # Confidence scoring + flag
│   │   ├── rag/
│   │   │   └── store.py          # ChromaDB: add_chunks, retrieve, stats
│   │   ├── engine/
│   │   │   └── config.py         # Env config, LLM client, embed_texts()
│   │   ├── ingest/
│   │   │   └── chunker.py        # Section-aware chunker 3GPP
│   │   ├── auth/
│   │   │   ├── security.py       # bcrypt, JWT create/verify
│   │   │   └── deps.py           # FastAPI dependency: get_current_user
│   │   └── db/
│   │       ├── database.py       # SQLAlchemy engine, Base, get_db
│   │       └── models.py         # ORM: User, Topic, ChatSession, Message
├── frontend/            # Next.js 14 (auth context, sources panel)
└── docker-compose.yml   # Postgres + backend + frontend

```

Lampiran B: Contoh Respons API

POST /api/query

Request:

```

{
  "topic_id": "<uuid-topik-ts24008>",
  "message": "Jelaskan prosedur GPRS Attach di TS 24.008",
  "top_k": 5
}

```

Response:

```

{
  "answer": "Prosedur GPRS Attach dimulai saat MS mengirim ATTACH REQUEST ke SGSN...",
  "confidence": 0.87,
  "flag": null,
  "verifier_verdict": "VALID",
  "verifier_reason": "Semua klaim didukung penuh oleh konteks §4.7.3.1",
  "regeneration_count": 0,
  "sources": [
    {
      "spec": "TS 24.008",
      "version": "g50",
      "section": "4.7.3",
      "title": "GPRS attach procedure",
    }
  ]
}

```

```
    "similarity": 0.9241,  
    "chunk_text": "[TS 24.008 vg50 §4.7.3] GPRS attach procedure\nThe GPRS attach..."  
  }  
],  
"query_id": "q_a1b2c3d4",  
"session_id": "<uuid-session>"  
}
```

Contoh Perilaku Anti-Halusinasi

Query: "Berapa harga iPhone terbaru?"

Response:

```
{  
  "answer": "Informasi tidak ditemukan dalam spec yang tersedia.",  
  "confidence": 0.0,  
  "flag": "NO CONTEXT",  
  "verifier_verdict": "INVALID",  
  "sources": [],  
  "regeneration_count": 0  
}
```